

Jane Street ASIC Puzzle 2026

Nishant Bhakar

August 7th, 2026

Abstract

We reverse engineer the ASIC by 1) recovering electrical wires from touching conductor polygons, 2) identifying standard cells joined by those wires, 3) translating the cells into Boolean equations, and 4) expanding the clocked circuit over time for Z3 to solve. We first validate this process on the warm-up and then apply it to the puzzle to retrieve the desired 121-bit input that, when written as an 11×11 Star Battle-inspired grid, has two stars per row, column, and region. Given this input, the ASIC prints `(* TWO STARS *)`.

1 Introduction

All computers deal in bits, and this ASIC is no different. The ASIC incrementally takes in 121 input bits and applies an incredibly deterministic procedure to them. We wish to find an input that turns on the `success` signal. This sounds like a Boolean satisfiability problem, and motivates our solution.

Directly solving the supplied ASIC is a daunting task, so instead we set out to solve the warm-up GDS, using its published behavior to test our process. Once our process is established, we repeat it with the ASIC.

2 The warm-up as a running example

The warm-up describes two 8-bit registers, a 9-bit adder, and a comparison with 496¹. On every rising clock edge for which `en=1`, 1 bit from A and 1 from B enter the two registers. After 8 edges, the register values are summed in the adder and the result is compared to 496, turning `S` on iff equal. This is succinctly described as

$$S = [a_{7:0} + b_{7:0} = 496],$$

where brackets mean a Boolean truth value. Figure 1 shows this circuit.

The sample tests shift bits in order from most significant to least significant. They first check a positive example, $250 + 246 = 496$, then a negative example, $1 + 2 \neq 496$. We shall use these values to test our recovered circuit.

¹Notably, an [even perfect number](#), and thus appearing as 111110000 in binary.

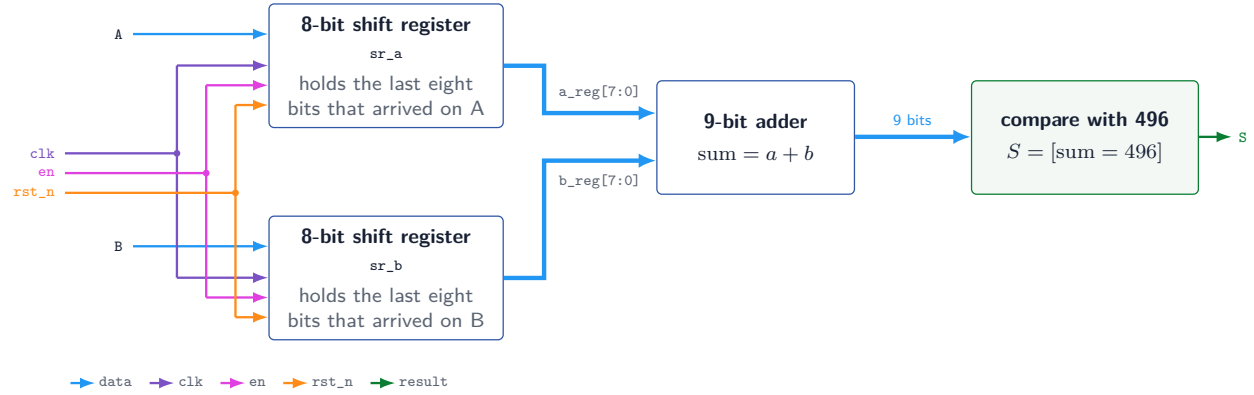


Figure 1: The warm-up circuit as described.

Listing 1: The warm-up behavior, abridged from warmup/00_source.v.

```

1 always @(posedge clk or negedge rst_n)
2   if (!rst_n) parallel_out <= 8'b0;
3   else if (en) parallel_out <= {parallel_out[6:0], serial_in};
4
5 assign sum = a + b;
6 assign eq  = (val == 9'd496);
7
8 comparator496 cmp0 (.val(sum), .eq(S));

```

3 From GDS geometry to wires

3.1 What the file describes

GDS stores polygons tagged with a layer/datatype pair, text labels, reusable cell definitions, and references that place copies of those definitions. This design uses the SkyWater SKY130 process and its high-density standard-cell library. In this process, layers 67–72 are local interconnect (LI1) through metal 5, datatype 20 denotes a conductor, and datatype 44 denotes a via cut. Identifying this geometry lets us recover the wires.

3.2 What the DSU represents logically

Our extractor gives an integer ID to each conductor polygon. A disjoint-set union (DSU) maintains groups of IDs. At first, every polygon is alone. We do `union(a,b)` if 1) polygons *a* and *b* intersect on the same conducting layer, or 2) a via cut intersects *a* and *b*. After all unions are completed, `find(a)` returns the same ID as `find(b)` iff *a* and *b* are connected through such contacts.

We treat each equivalence class as one ideal electrical node, called a net. Note that the DSU records only connectivity; it does not record logical operations, time, direction, or a signal's value.

Listing 2: The 2 physical rules to extract the netlist from GDS, abridged from our extractor.

```

1 # Same mask layer: exact polygon contact is electrical contact.
2 for i, polygon in enumerate(layer.polygons):
3     for j in layer.tree.query(polygon, predicate="intersects"):
4         dsu.union(layer.offset + i, layer.offset + int(j))
5
6 # A via cut joins every conductor it touches below to every one above.
7 for cut in via_cuts:
8     for i in conductors_below_touching(cut):
9         for j in conductors_above_touching(cut):
10            dsu.union(below.offset + i, above.offset + j)

```

In the worked example below, the DSU groups 13 conductor polygons on 3 layers through 8 same-layer contacts and 4 via cuts, and attaches 2 cell endpoints. The 3 layers are drawn separately in Figure 2 to make the contacts visible.

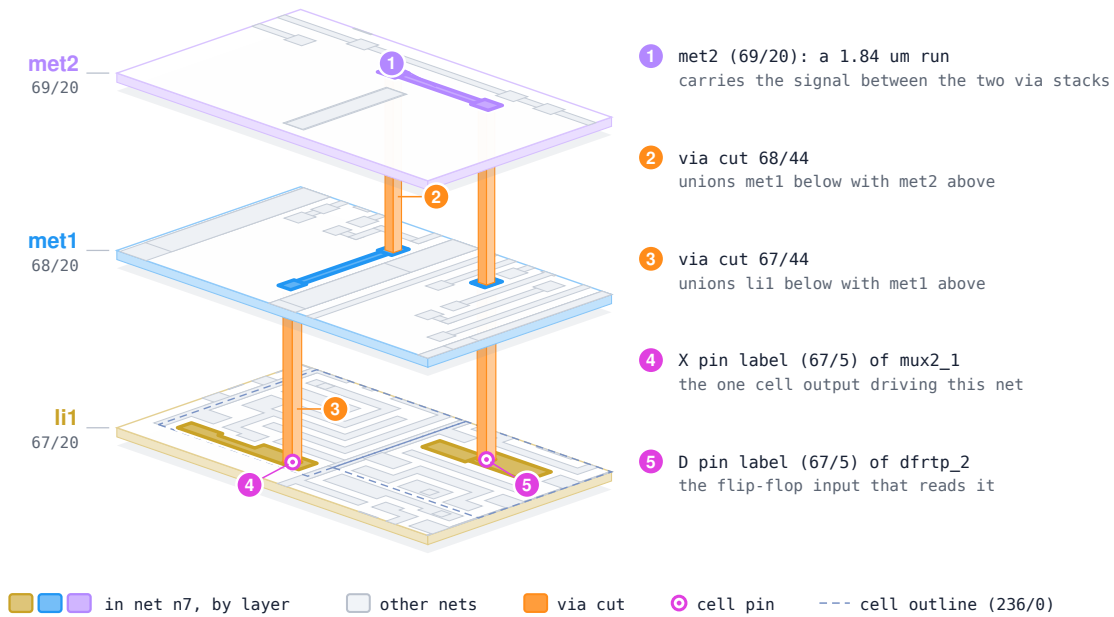


Figure 2: Recovered warm-up net **n7**.

3.3 Attaching names to components

Standard-cell definitions contain pin labels on LI1. A placed reference carries a transform: a location, a rotation, and possibly a reflection. We apply that transform to the label coordinate and extract which DSU component lies under that point. For example,

$$\text{sr_a/_11_}.X \mapsto n7$$

means that the transformed coordinate of pin label **X** lies on a conductor polygon whose DSU root was compactly renamed **n7**. The compact IDs are arbitrary; the endpoint grouping is what matters.

Repeating this for every pin label in the design gives the netlist: a list of placed cells together with the net attached to each of their pins.

4 From named cells to Boolean equations

Connectivity tells us which pins share a wire, but cell semantics tell us what those pins compute.

4.1 Standard cells, masters, and instances

This design does not lay out transistors gate by gate. It assembles circuits from small, pre-designed components such as AND gates, multiplexers, and flip-flops, each stored once in the GDS as a definition and then placed wherever it is needed. Those definitions have names beginning `sky130_fd_sc_hd__`.

For example, one embedded definition is named

`sky130_fd_sc_hd__mux2_1.`

The `mux2` identifies a 2-input multiplexer, and the `1` identifies its drive-strength variant. Drive strength does not affect the Boolean rule, so we will ignore it. We call a reusable GDS definition a master; an instance is one such placed copy of it. In the warm-up, `sr_a/_11_` is an instance of the `mux2_1` master.

The master defines 4 signal pins:

Pin	Direction	Meaning
A0	input	value selected when $S=0$
A1	input	value selected when $S=1$
S	input	selection bit
X	output	selected value

Consequently, `sr_a/_11_.X` means “pin X on this particular placed mux.”

4.2 Substituting nets into a documented cell rule

Our geometric reconstruction attaches the 4 pins of this instance to the following nets:

$\text{sr_a/_11_.A0} \mapsto n28,$
 $\text{sr_a/_11_.A1} \mapsto n79,$
 $\text{sr_a/_11_.S} \mapsto n80,$
 $\text{sr_a/_11_.X} \mapsto n7.$

Each net carries a Boolean value. The 3 input pins read the values on $n28$, $n79$, and $n80$; the output pin drives the result onto $n7$. A mux operates under the following rule:

$$X = \begin{cases} A0, & S = 0, \\ A1, & S = 1. \end{cases} \iff n7 = \begin{cases} n28, & n80 = 0, \\ n79, & n80 = 1. \end{cases}$$

Equivalently,

$$n7 = \text{ite}(n80, n79, n28) = (n80 \wedge n79) \vee (\neg n80 \wedge n28),$$

where $\text{ite}(c, x, y)$ means “if c , then x , otherwise y .”

In this example, $n80$ is connected to the top-level input $\mathbf{en}$. When $\mathbf{en}=1$, the mux passes $n79$, the output of the preceding shift-register stage, toward the next flip-flop. When $\mathbf{en}=0$, it passes $n28$ back, which is that flip-flop's own output, so the flip-flop retains its current value at the next clock edge.

4.3 Translating the remaining cells

We repeat the same substitution for every placed component. For example, a 2-input AND cell obeys

$$X = A \wedge B.$$

If one particular instance has $A \rightarrow n3$, $B \rightarrow n12$, and $X \rightarrow n25$, that instance contributes

$$n25 = n3 \wedge n12.$$

An inverter contributes a negation. More complicated cells contribute larger expressions. For example, the documented function of an `a21o` cell is

$$X = (A1 \wedge A2) \vee B1.$$

This process produces Boolean equations for all combinational cells: components whose outputs depend only on their current inputs. Flip-flops require one additional idea because their outputs depend on an earlier clock edge.

5 Adding memory and the clock

A combinational equation relates values at a specific point in time. A flip-flop introduces state. Specifically, an output Q after a rising edge equals D just before that edge. We therefore index state by clock step. If $n28$ is the Q output of `sr_a/_19_`, then

$$n28_{t+1} = \begin{cases} 0, & \text{if reset is active,} \\ n7_t, & \text{otherwise.} \end{cases}$$

Substituting the mux equation yields

$$n28_{t+1} = \text{ite}(\mathbf{en}_t, n79_t, n28_t) \quad \text{when reset is inactive.}$$

Figure 3 illustrates this rule, both as a circuit and over time.

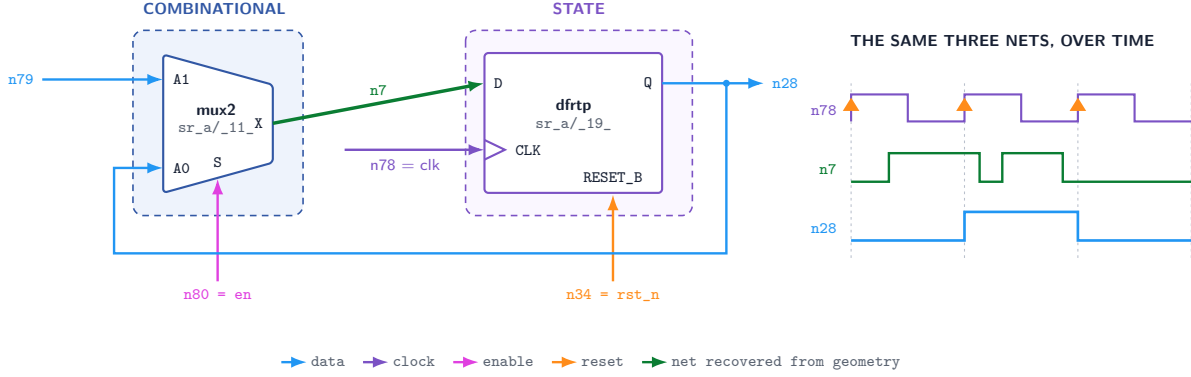


Figure 3: One mux-to-flip-flop path, shown as a circuit and over time.

The mux output $n7$ changes whenever its inputs change. The flip-flop copies $n7$ into $n28$ only at a rising clock edge, then holds it. Thus the clock marks when the transition equation applies. Pulling reset low overrides the clock and forces $n28$ to 0.

6 Turning a clocked circuit into one finite formula

We give the state vector a separate name at each modeled clock edge and connect adjacent states using the next-state function. We call this unrolling the circuit. This leaves one finite combinational formula.

For the warm-up, applying the transition rule 8 times creates states $Q_0, Q_1, \dots, Q_8$. Reset fixes Q_0 ; the serial input bits at each step are known values; and we evaluate the final expression for S . Because nothing here is unknown, the warm-up is a check rather than a search: S comes out 1 for $250 + 246$ and 0 for $1 + 2$, exactly as the published sample tests require.

The warm-up also ships its routed DEF, which states the true connectivity, so we can compare that against our recovered nets. All 84 signal nets and all 291 endpoints agree: no net is missing an endpoint, none has gained one, none is split into two, and no two are merged into one. Since the recovered circuit agrees with both the sample tests and the DEF, we use the same method on the ASIC.

The ASIC uses exactly this construction, with one change: the serial inputs are unknown Boolean variables rather than known bits. Let Q_t be the vector of all 92 flip-flop outputs immediately after edge t , and let I_t be the bit presented at the next edge. The transition function T evaluates every recovered combinational equation using Q_t , I_t , and enable_t , then collects the resulting flip-flop outputs. Thus

$$Q_{t+1} = T(Q_t, I_t, \text{enable}_t).$$

In the supplied waveform, `example_inputs.vcd`, there are 121 transitions with `enable=1`, followed by 1 with `enable=0`. Therefore, there are states Q_0 through Q_{122} , with the goal being $\text{success}(Q_{122}) = 1$. Figure 4 shows the resulting chain.

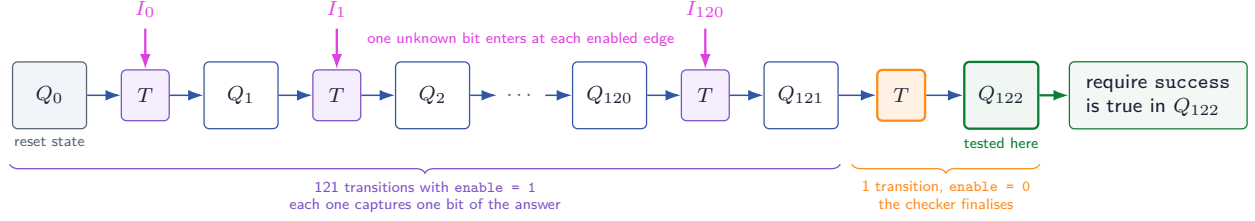


Figure 4: The puzzle unrolled across 122 clock edges.

The extra disabled transition is necessary because **success** is stored only after input capture ends.

A Boolean satisfiability (SAT) solver takes a set of constraints over Boolean variables and either returns an assignment that satisfies all of them at once, called a model, or reports **unsat**, meaning it has proved that no such assignment exists. That is exactly the problem promised in the introduction, so we hand our equations to Z3.

The exported instance contains 11,437 variables: 121 unknown inputs and $92 \times 123 = 11,316$ state variables. It contains 11,313 top-level assertions: 88 reset facts, 92×122 transition equalities, and 1 final success requirement. The other 4 flip-flops have no reset pin. Under the recovered transition relation, a backward fan-in audit shows that none lies in the transitive fan-in of **success**, so their unconstrained initial values do not affect the result.

To prove uniqueness, after the first model $b_0, \dots, b_{120}$ is found, we add the following clause:

$$(I_0 \neq b_0) \vee \dots \vee (I_{120} \neq b_{120}).$$

It is false for exactly the assignment just found and true for every other 121-bit string, so solving again searches for a different input. Z3 returns **unsat**, which proves that no other 121-bit input satisfies the recovered circuit.

7 Result

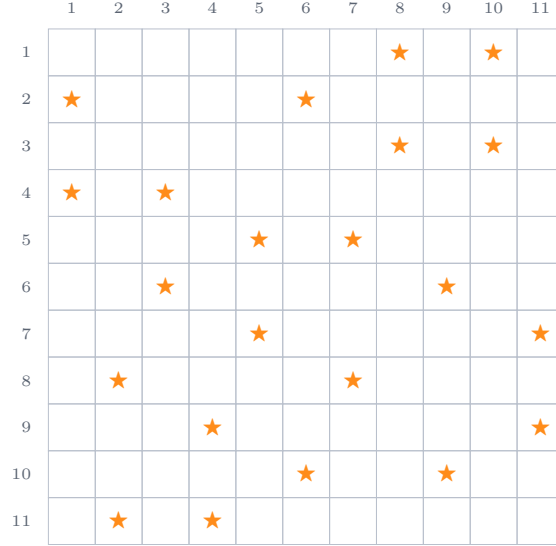
The unique 121-bit input is

```
0000000010101000010000000000000101010100000000000010100000010000001000000101000010
000000100000010000010010001010000000
```

On the next rising edge, **success** becomes true, and the output circuit emits

(* TWO STARS *)

Arranged in the serial stream's row-major order on an 11×11 grid, the input has exactly two 1-bits in every row and column, and no two 1-bits touch horizontally, vertically, or diagonally (Figure 5). This is oddly reminiscent of Star Battle, leading us to ponder what the mystery regions are.

Figure 5: The recovered input in an 11×11 grid.

7.1 Recovering the regions

Closer inspection of the 92 flip-flops reveals that 44 lie in 2 regular banks of 22. Tracing their next-state equations pairs each bank into eleven 2-bit counters. One bank is selected by column; the other is selected by an irregular position-dependent signal. Because the 121 bits arrive serially in row-major order, the same 11-column selection schedule is reused for every row.

For a concrete example, one pair has low bit **n717** and high bit **n509**. Simplifying the gates that drive their D pins gives the following behavior, with x meaning the current input is 1, capture is enabled, and this particular counter is selected:

x	old HL	new HL	meaning
0	any	unchanged	this cell does not add to the counter
1	00	01	first selected star
1	01	10	second selected star
1	10 or 11	11	third or later star, saturated at 3

Thus the remaining problem is to learn which counter is selected at each of the 121 positions. We reset the circuit and feed 0 for every position before t . The position machinery still advances, so the selection signals are correct for position t , but no star has been counted anywhere and every counter therefore sits at 00. From 00, a selected counter must move to 01 when it sees a star, while an unselected one cannot move at all. So we copy that pre-edge state and evaluate the next state twice, once with $I_t = 0$ and once with $I_t = 1$. For these 2 counter banks, exactly 1 low flip-flop per bank differs between the 2 evaluations, and it is the low bit of the counter selected at position t .

For cell $(1, 1)$, the differing low bits are **n313** in one bank and **n717** in the other. The **n313** choice recurs at cells $(2, 1), (3, 1), \dots$, identifying column 1. At $(1, 2)$ the column bit changes to **n304**, while the region bit is still **n717**, so $(1, 1)$ and $(1, 2)$ must share the same region. At $(1, 6)$ the pair is **n671** and **n495**: column 6 and a new region. Repeating this comparison for all 121 stream positions yields the complete partition.

The extracted schedule must pass 4 checks: 11 column selectors repeat every row; 11 region selectors cover all 121 cells exactly once; each resulting region is orthogonally connected; and the winning replay leaves every column and region counter at binary 10. All 4 hold, and the resulting reconstruction is Figure 6.

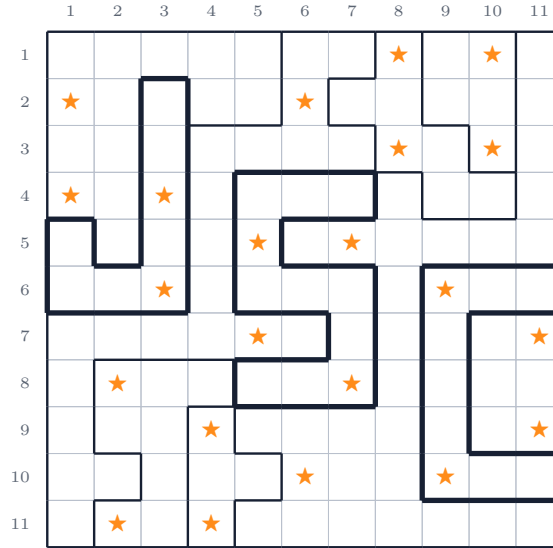


Figure 6: The recovered board and its region boundaries.

The ASIC is therefore fully accounted for. Touching polygons gave us wires, wires plus the cell library gave us Boolean equations, the clock turned those equations into one unrolled formula, and that formula had exactly one solution: the unique Star Battle board above, which the ASIC confirms by printing (`* TWO STARS *`).

8 AI policy

No AI system solved any part of this puzzle or contributed to the paper’s writing. The geometry rules, netlist reconstruction, cell semantics, region recovery, analysis, and conclusions are entirely my own. AI was used only as a coding assistant to implement supporting tools from specifications I had already established.

9 Easter eggs

1. The two attempts in `example_inputs.vcd` encode *The night sky awaits*: split each into eleven 11-bit records, reverse each record’s first 7 bits, and read only those 7-bit character codes (ignoring 4 zero padding bits; the last 2 codes are spaces).
2. Below the die, 36 inert tiny-cell references spell out PER ARENAM AD ASTRA in Morse code, “through the arena to the stars,” a spin on *per aspera ad astra*.
3. The output circuit has 4 reachable responses: EMPTY SKY for all 0s, BIG BANG for all 1s, TRY AGAIN for an ordinary incorrect input, and, naturally following OCaml comment syntax, (`* TWO STARS *`) for the answer.